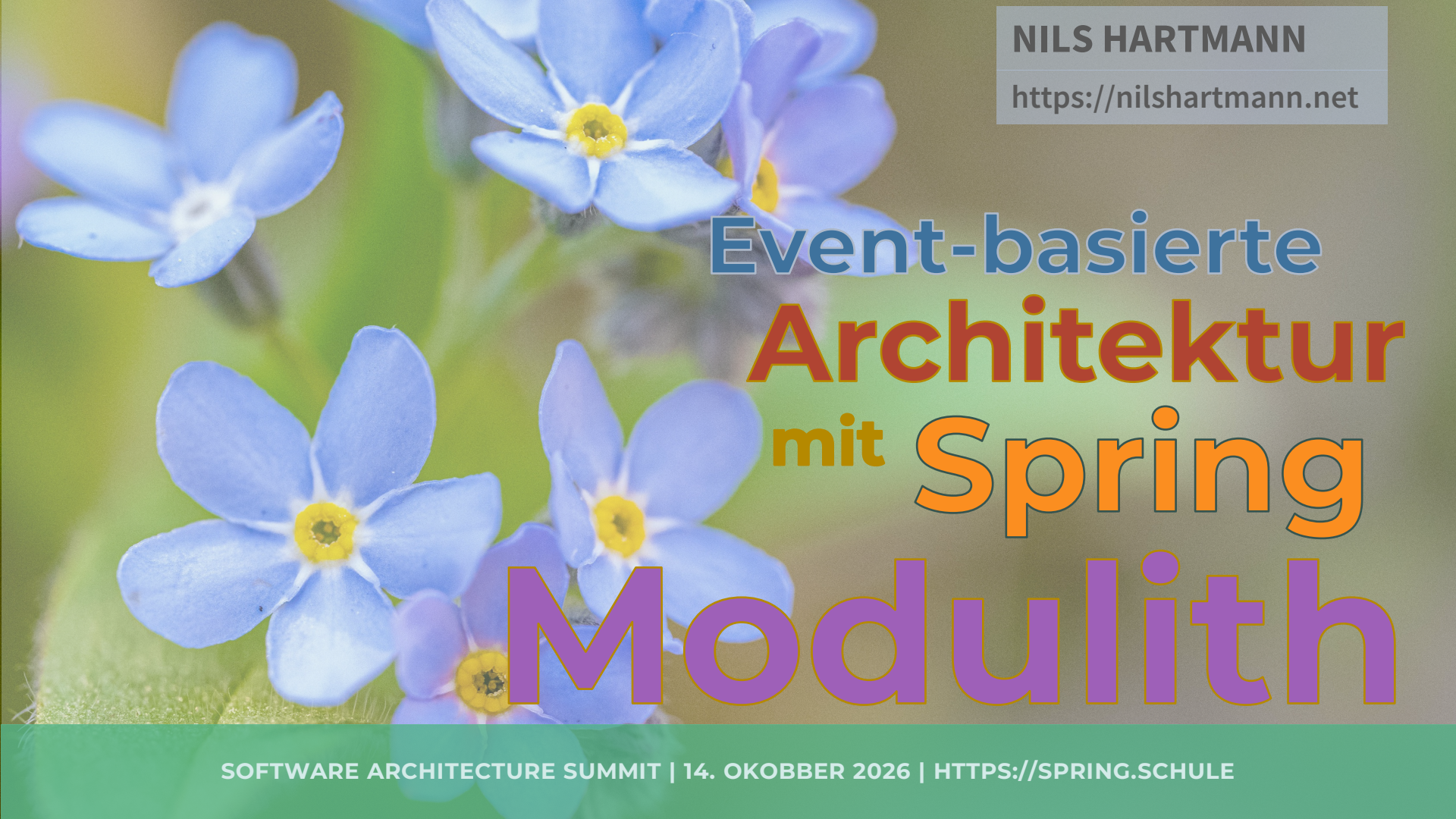


NILS HARTMANN

https://nilshartmann.net

Event-basierte Architektur mit Spring Modulith

SOFTWARE ARCHITECTURE SUMMIT | 14. OKOBBER 2026 | HTTPS://SPRING.SCHULE

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

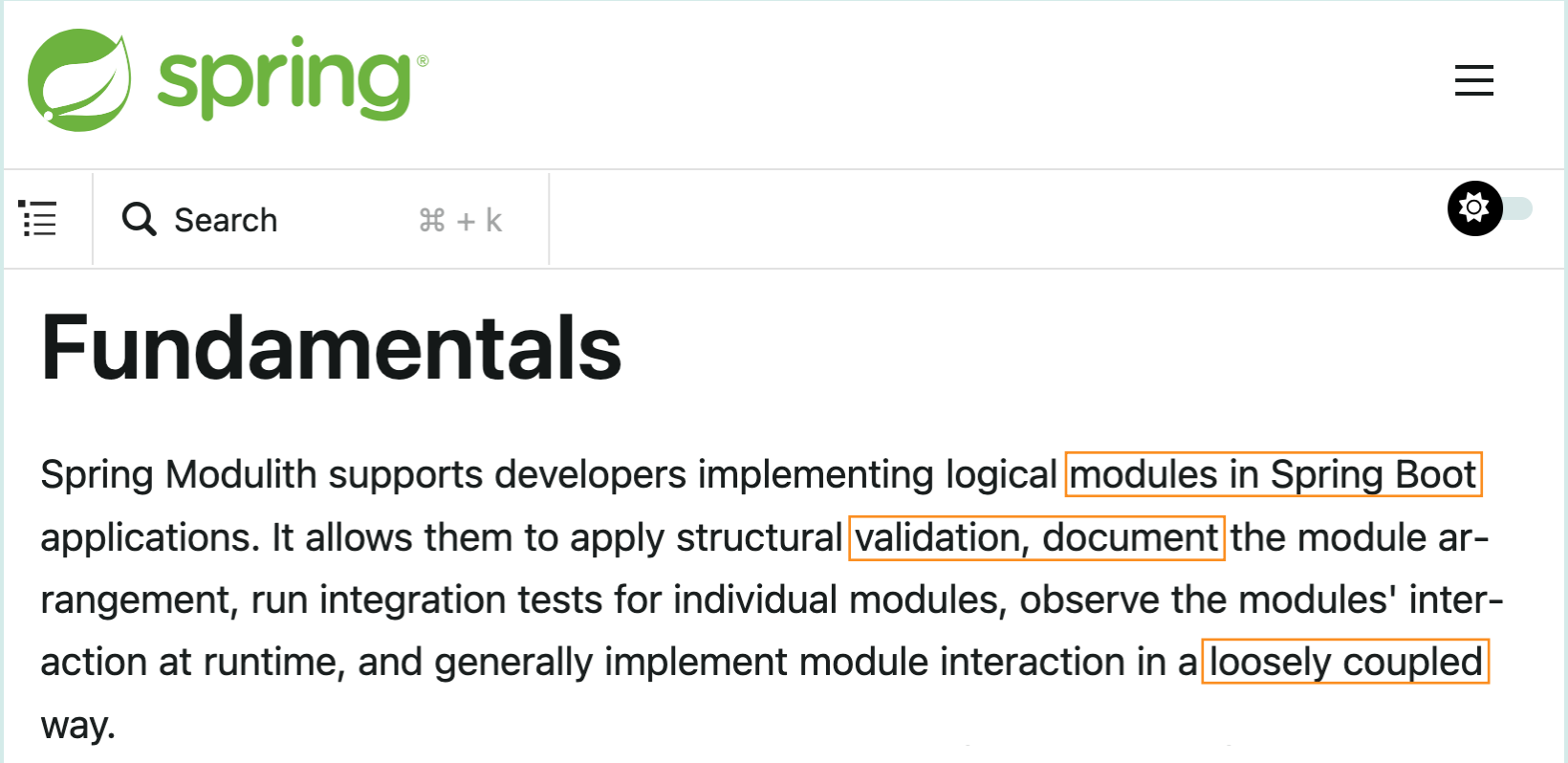
Java, Spring, GraphQL, React, TypeScript



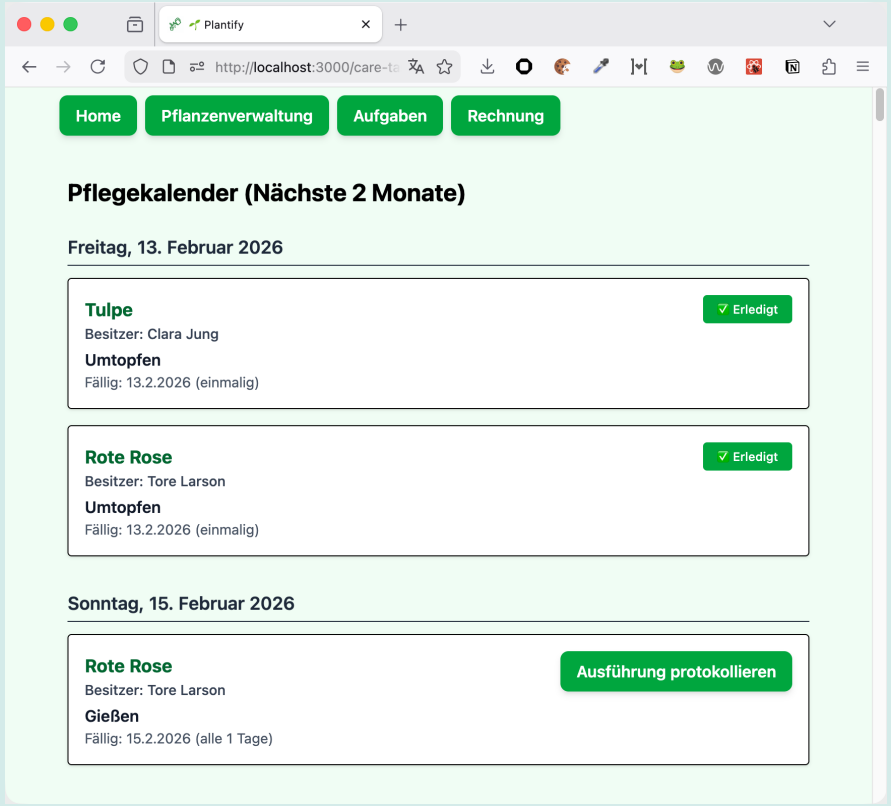
https://react.schule/video-kurs-react



https://reactbuch.de



https://docs.spring.io/spring-modulith/reference/fundamentals.html


Plantify
http://localhost:3000/care-ta
Home
Pflanzenverwaltung
Aufgaben
Rechnung
Pflegekalender (Nächste 2 Monate)
Freitag, 13. Februar 2026
Tulpe
Besitzer: Clara Jung
Umtopfen
Fällig: 13.2.2026 (einmalig)
Erledigt
Rote Rose
Besitzer: Tore Larson
Umtopfen
Fällig: 13.2.2026 (einmalig)
Erledigt
Sonntag, 15. Februar 2026
Rote Rose
Besitzer: Tore Larson
Gießen
Fällig: 15.2.2026 (alle 1 Tage)
Ausführung protokollieren

BEISPIEL

MODULITHEN

MODULITHEN

- **Monolith**

Monolith

- **Eine** Codebasis, **ein** Deployment
- Gefahr: Keine klare Struktur, alles hängt an allem
- Gefahr: hohe Kopplung entsteht, sinkende Wartbarkeit

- **Monolith vs. Microservices**

Monolith

- **Eine** Codebasis, **ein** Deployment
- Gefahr: Keine klare Struktur, alles hängt an allem
- Gefahr: hohe Kopplung entsteht, sinkende Wartbarkeit

Microservices

- Viele **fachliche Services**
- Lose Kopplung zwischen **Services**
- Maximale Unabhängigkeit (individuelle Deployments, Programmiersprache, Datenhaltung, ...)
- aber hoher operativer Aufwand

MODULITHEN

- **Monolith vs. Microservices vs. Modulith**

Monolith

- **Eine** Codebasis, **ein** Deployment
- Gefahr: Keine klare Struktur, alles hängt an allem
- Gefahr: hohe Kopplung entsteht, sinkende Wartbarkeit

Modulith

(modularer Monolith)

- **Eine** Codebasis, **ein** Deployment
- Klare **fachliche Module**
- Lose Kopplung zwischen **Modulen**

Microservices

- Viele **fachliche Services**
- Lose Kopplung zwischen **Services**
- Maximale Unabhängigkeit (individuelle Deployments, Programmiersprache, Datenhaltung, ...)
- aber hoher operativer Aufwand

MODULITHEN

- **Monolith vs. Microservices vs. Modulith**

Monolith

- **Eine** Codebasis, **ein** Deployment
- Gefahr: Keine klare Struktur, alles hängt an allem
- Gefahr: hohe Kopplung entsteht, sinkende Wartbarkeit

Modulith

(modularer Monolith)

- **Eine** Codebasis, **ein** Deployment
- Klare **fachliche Module**
- Lose Kopplung zwischen **Modulen**

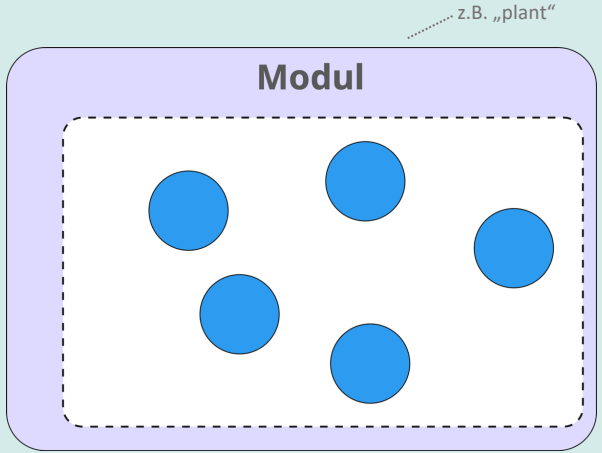
Microservices

- Viele **fachliche Services**
- Lose Kopplung zwischen **Services**
- Maximale Unabhängigkeit (individuelle Deployments, Programmiersprache, Datenhaltung, ...)
- aber hoher operativer Aufwand

👉 ***Modulith**: Monolith, aber mit Struktur, ohne Microservice Kosten*

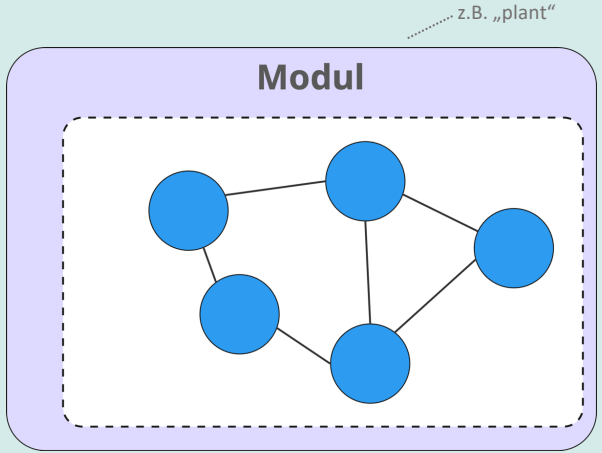
MODULE

- Klare fachliche Verantwortung, in-sich abgeschlossen



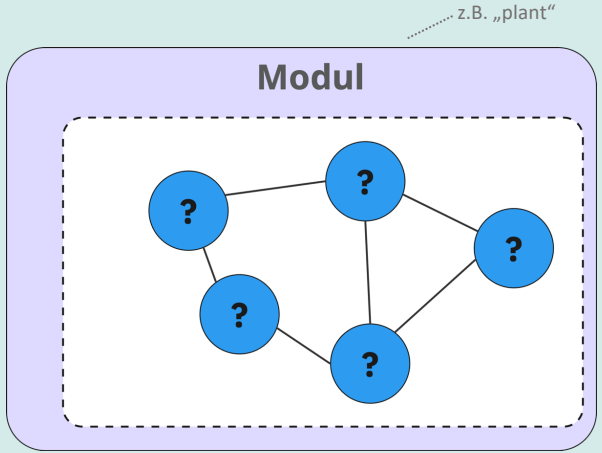
MODULE

- Klare fachliche Verantwortung, in-sich abgeschlossen
- Hohe Kohäsion innerhalb



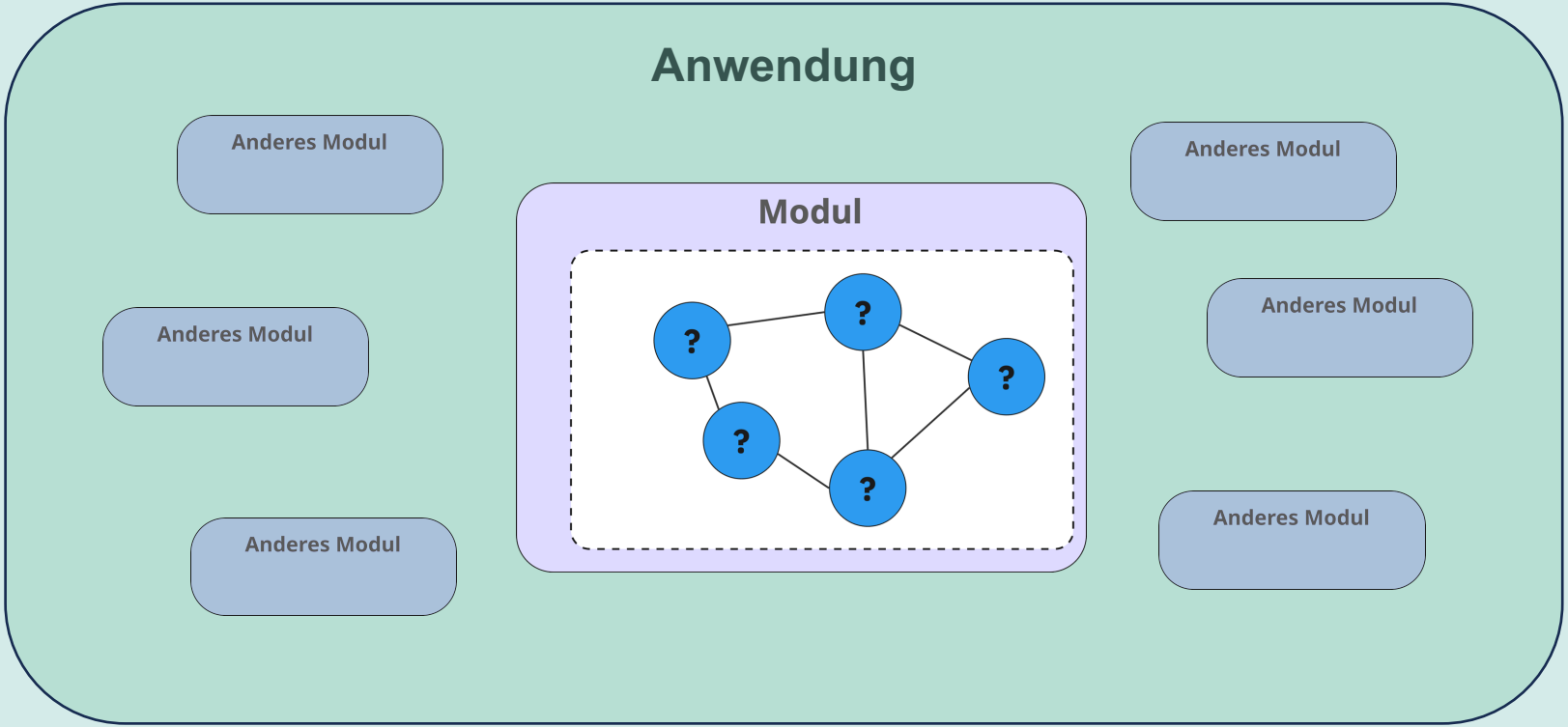
MODULE

- Klare fachliche Verantwortung, in-sich abgeschlossen
- Hohe Kohäsion innerhalb
- Innenleben bleibt verborgen



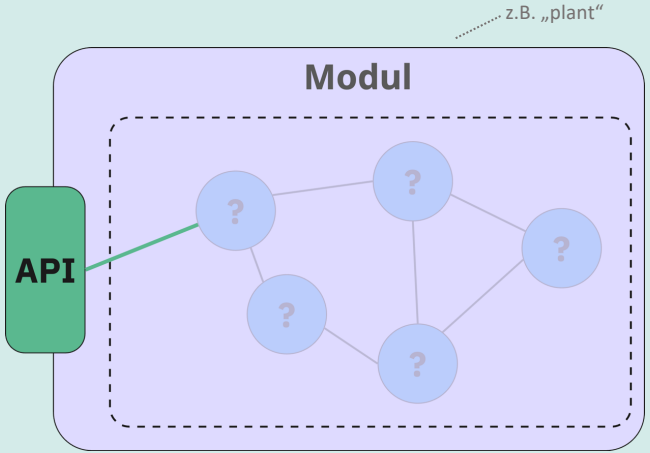
MODULE

- Leben nicht in Isolation...



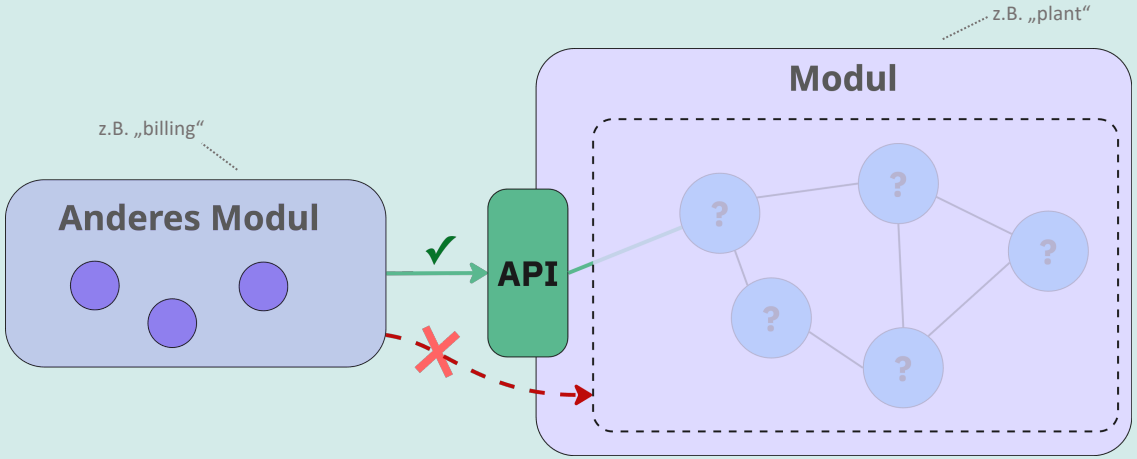
MODULE

- Nach außen: explizite Schnittstelle
-



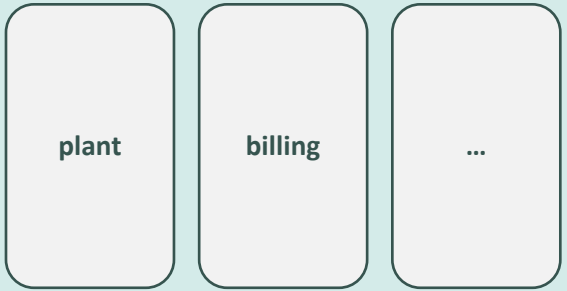
MODULE

- Nach außen: explizite Schnittstelle
- Lose Kopplung
-



Modulith

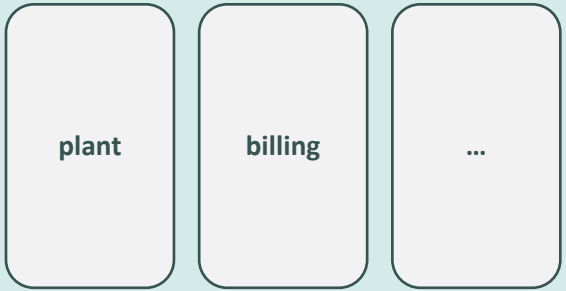
- Gliedert die Anwendung in vertikal-fachliche Module



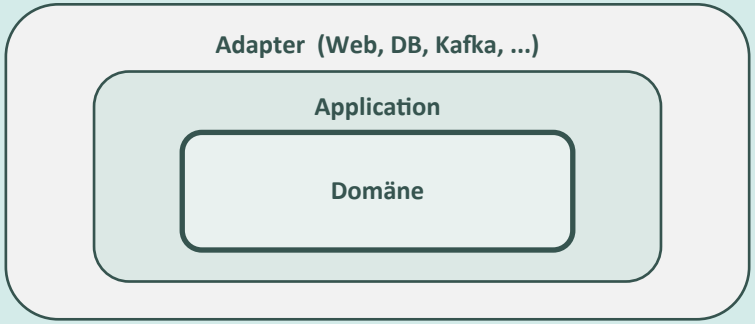
Modulith
vertikale, fachliche Module

Hexagonale Architektur

- Strukturierung durch horizontal-technische Ebenen



Modulith
vertikale, fachliche Module

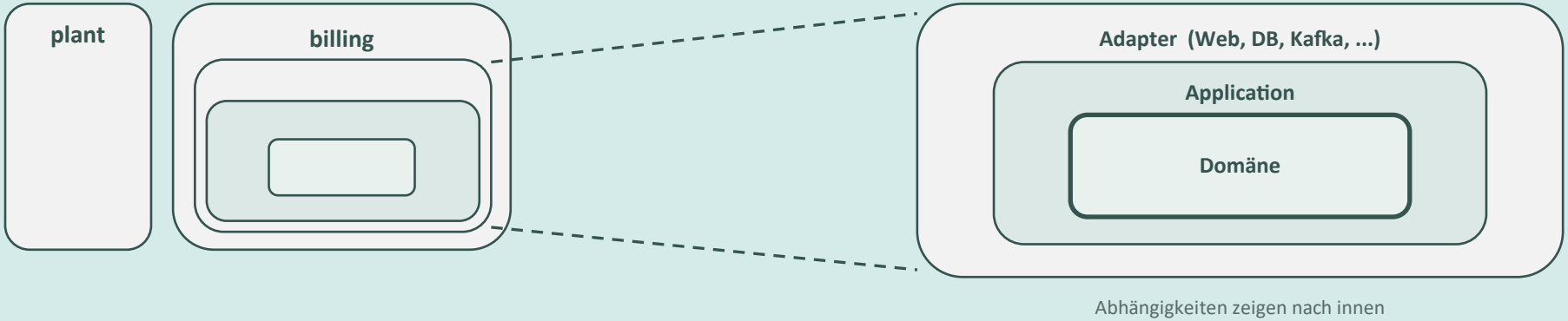


Abhängigkeiten zeigen nach innen

Hexagonal (Ports & Adapters)
technische Schichten

Hexagonale Architektur

- Strukturierung durch horizontal-technische Ebenen
- Kann interne Struktur eines Moduls vorgeben

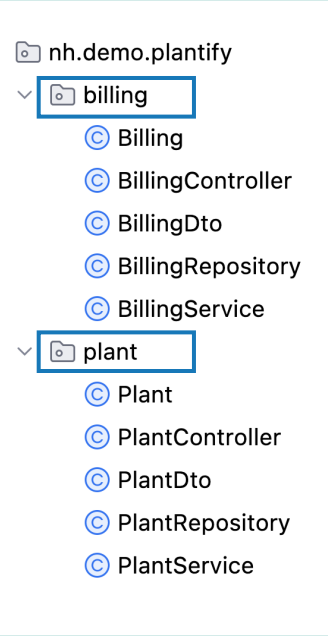


Modulith
vertikale, fachliche Module

Hexagonal (Ports & Adapters)
technische Schichten

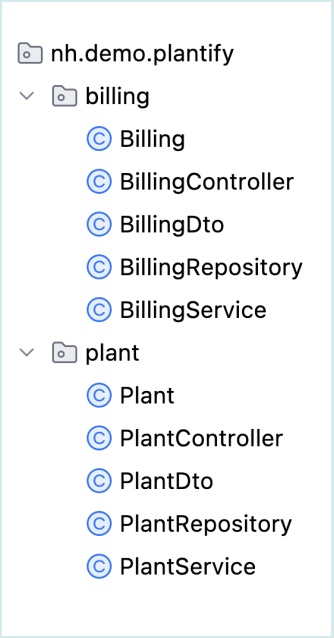
Spring Modulith: Gliederung der Anwendung in Module

Spring Modulith: Gliederung der Anwendung in Module



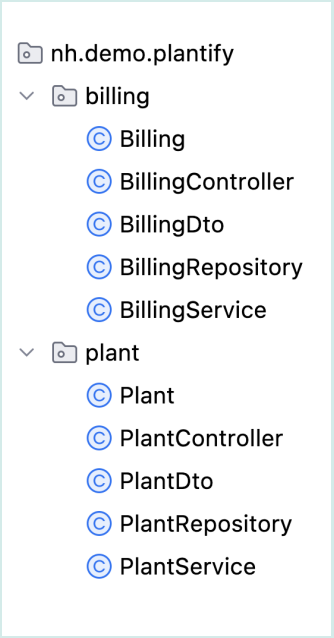
- Packages sind *Module*

Gliederung der Anwendung in Module



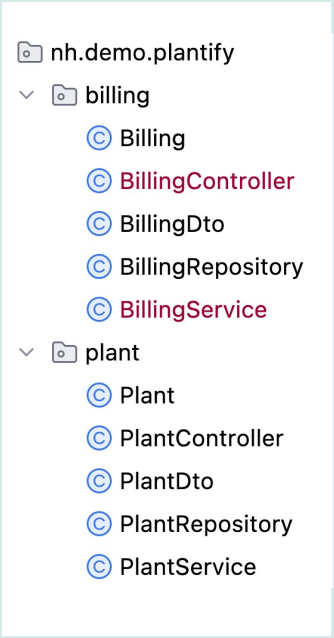
- Fachlichkeit ist fachlichen Packages konzentriert

Gliederung der Anwendung in Module



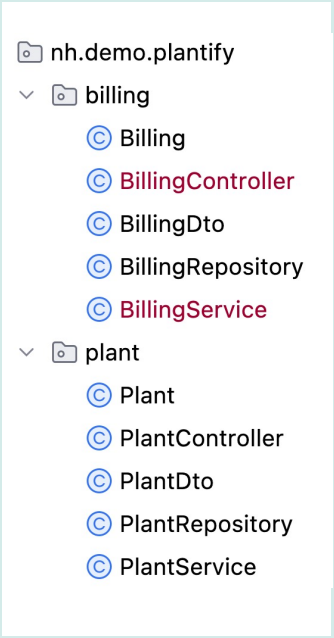
- Fachlichkeit ist fachlichen Packages konzentriert
- Vertikaler „Slice“, enthält alle Schichten

Gliederung der Anwendung in Module



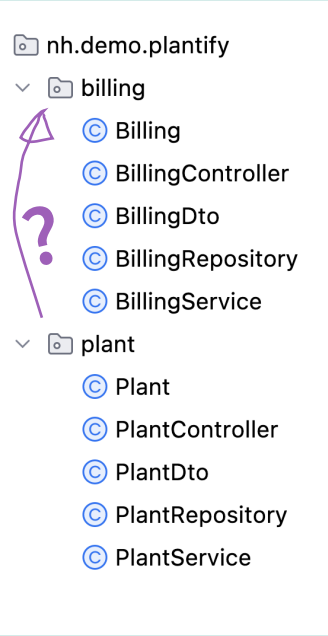
- Fachlichkeit ist fachlichen Packages konzentriert
- Vertikaler „Slice“, enthält alle Schichten
- Änderungen müssen nur an einer Stelle gemacht werden

Gliederung der Anwendung in Module



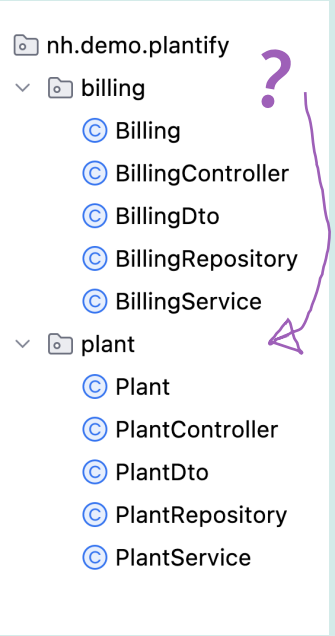
- Fachlichkeit ist fachlichen Packages konzentriert
- Vertikaler „Slice“, enthält alle Schichten
- Änderungen müssen nur an einer Stelle gemacht werden
- Hohe Kohäsion

Gliederung der Anwendung in Module



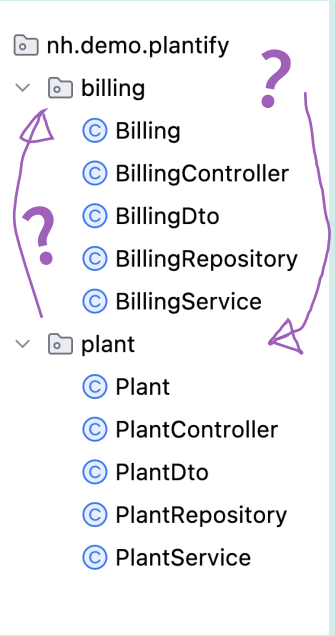
- Problem: Wie sehen Abhängigkeiten aus?

Gliederung der Anwendung in Module



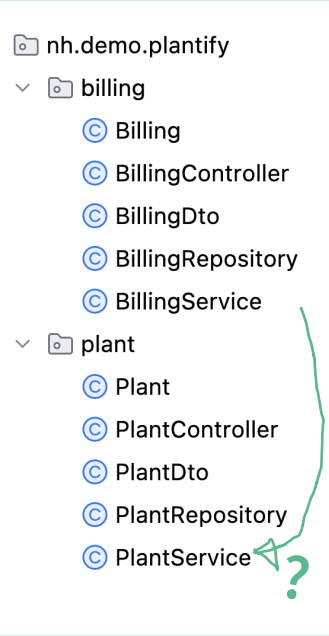
- Problem: Wie sehen Abhängigkeiten aus?
- Oder so?

Gliederung der Anwendung in Module



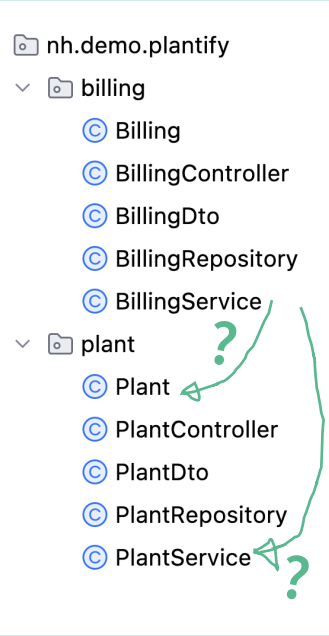
- Problem: Wie sehen Abhängigkeiten aus?
- Oder so?
- Mal so, mal so 🤔

Gliederung der Anwendung in Module



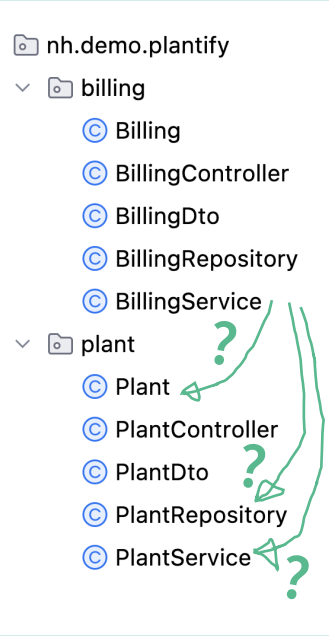
- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔

Gliederung der Anwendung in Module



- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔
- 🤔 🤔

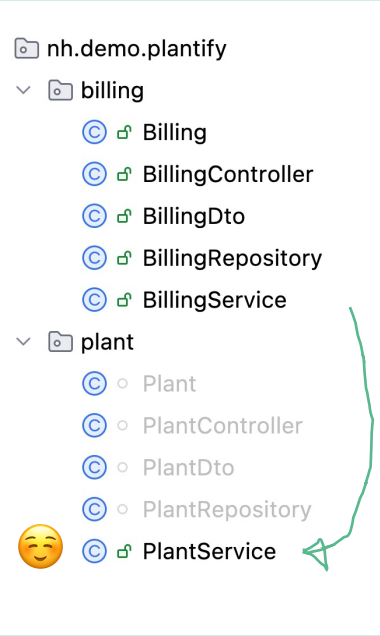
Gliederung der Anwendung in Module



- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔
- 🤔🤔
- 🤔🤔🤔

Gliederung der Anwendung in Module

- Problem: Wie sieht die **Schnittstelle** von „plant“ aus?



Sichtbarkeit von Klassen einschränken

```java
package nh.demo.plantify.plant;

// Öffentlich
public  class PlantService {

    // ...

}
```

✅

```java
package nh.demo.plantify.plant;

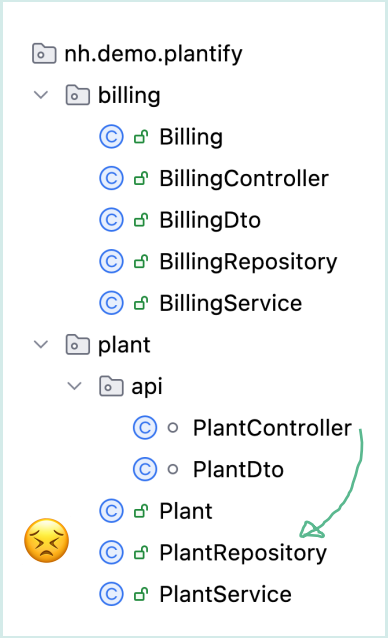
// Nur Plant-intern
class PlantRepository  {

    // ...

}
```

⛔

Gliederung der Anwendung in Module



Sichtbarkeit von Klassen ~~einschränken~~

```java
package nh.demo.plantify.plant;

// Öffentlich
public  class PlantService {

    // ...

}
```

✅

👀

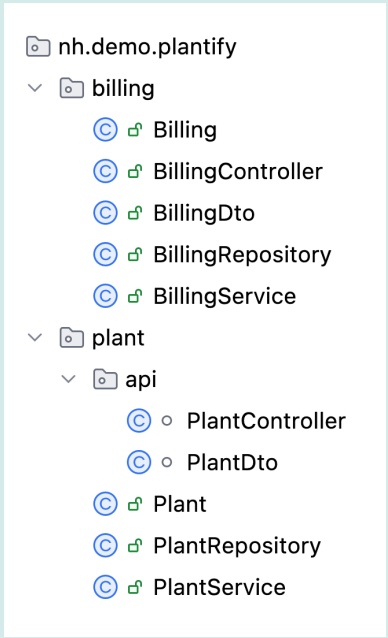
```java
package nh.demo.plantify.plant;

// Nur Plant-intern
public class PlantRepository  {

    // ...

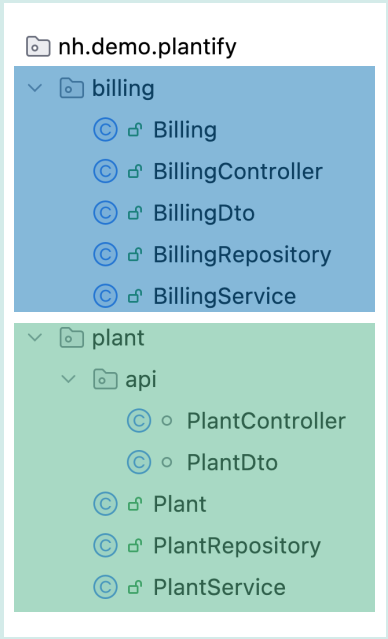
}
```

Modularisierung mit Java-Bordmitteln



Was fehlt?

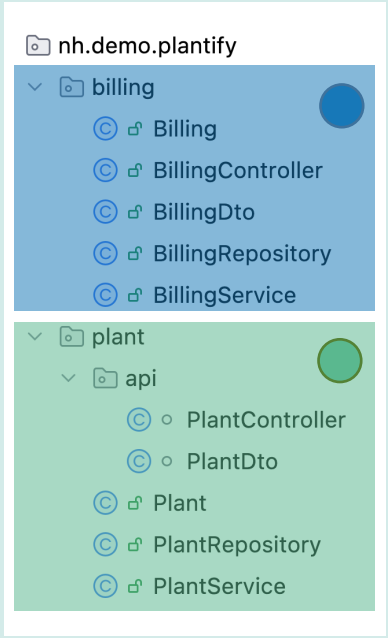
Modularisierung mit Java-Bordmitteln



Was fehlt?

- „Echte“ Module
  - Mehr als Klassen oder Packages

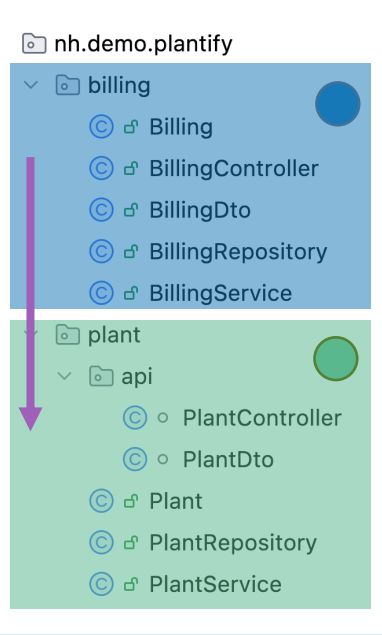
Modularisierung mit Java-Bordmitteln



Was fehlt?

- **„Echte“ Module**
  - Mehr als Klassen oder Packages
- **Klare Schnittstellen**
  - Inneres verbergen

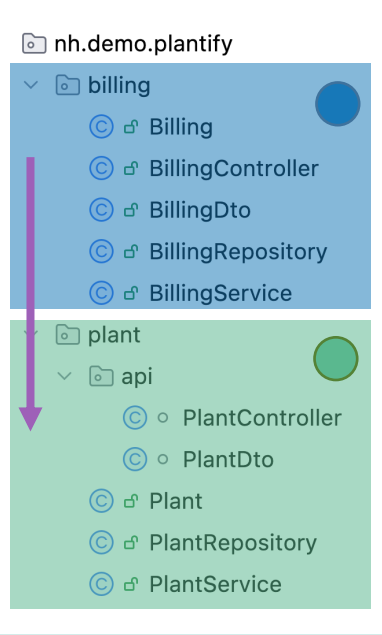
Modularisierung mit Java-Bordmitteln



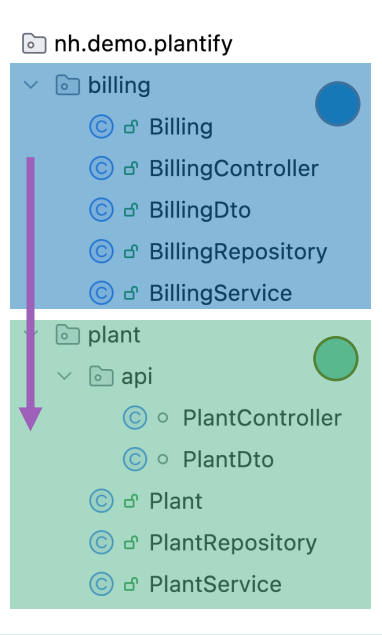
Was fehlt?

- **„Echte“ Module**
  - Mehr als Klassen oder Packages
- **Klare Schnittstellen**
  - Inneres verbergen
- **Explizite Abhängigkeiten**
  - Zwischen den Modulen

- Spring Modulith

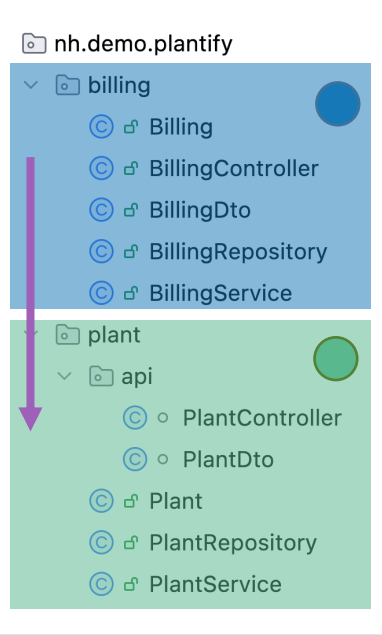


- **Spring Modulith**



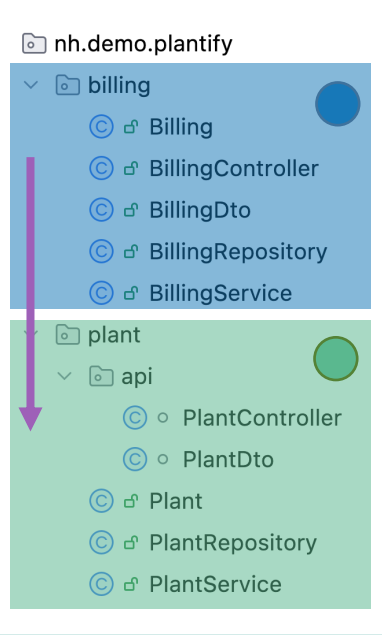
- Packages werden zu *Application Modules*

- **Spring Modulith**

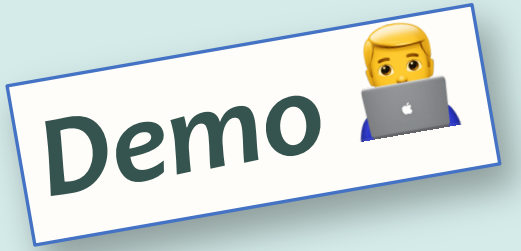


- **Packages werden zu *Application Modules***
  - Direkte Packages unterhalb des main-Packages
  - Definierte Abhängigkeiten
  - Explizite Schnittstellen

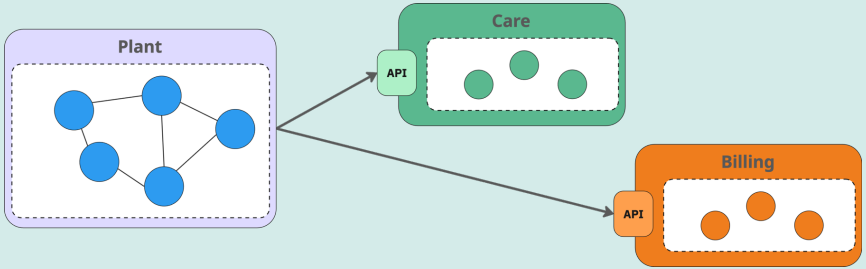
- **Spring Modulith**



- Packages werden zu *Application Modules*
  - Direkte Packages unterhalb des main-Packages
  - Definierte Abhängigkeiten
  - Explizite Schnittstellen

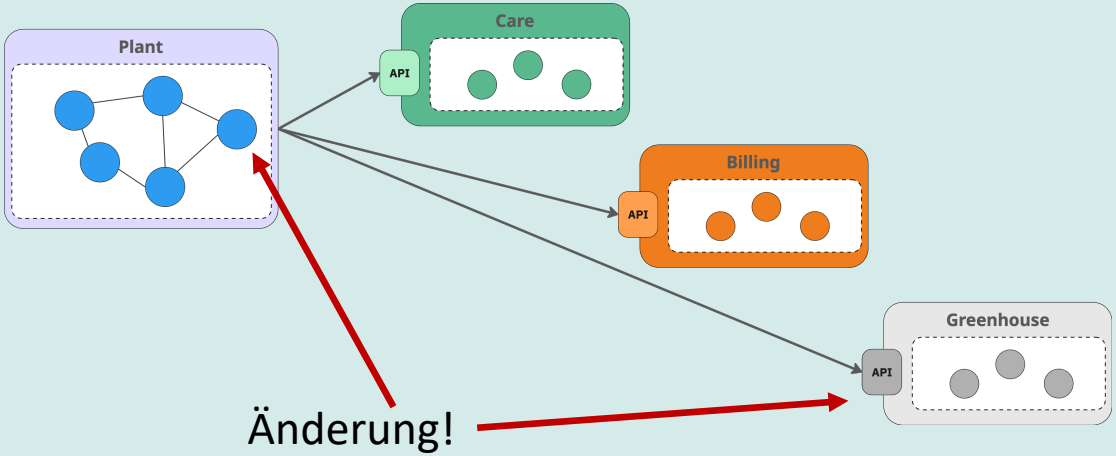


Kopplung von Modulen über Services



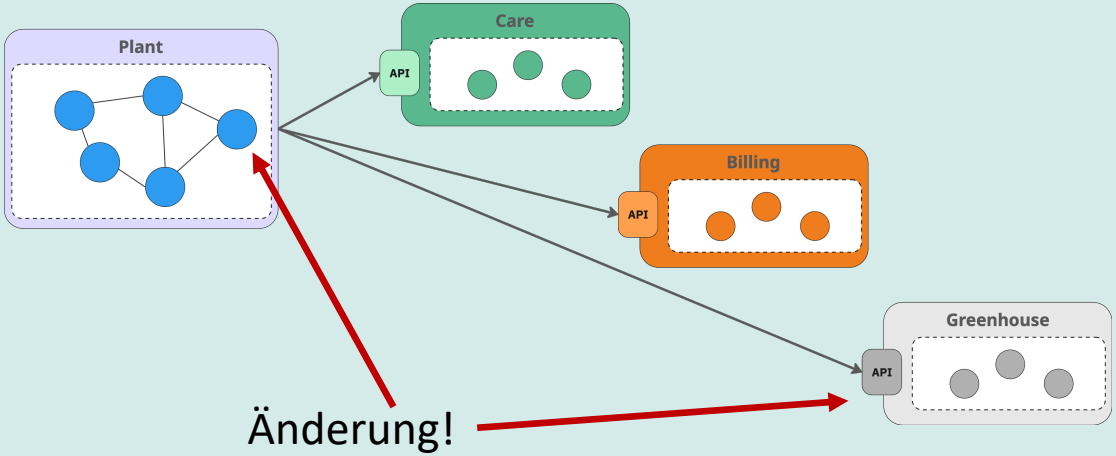
Kopplung von Modulen über Services

- Erweiterungen müssen an zwei Stellen erfolgen
- Tests müssen angepasst werden (Beans in Services!)



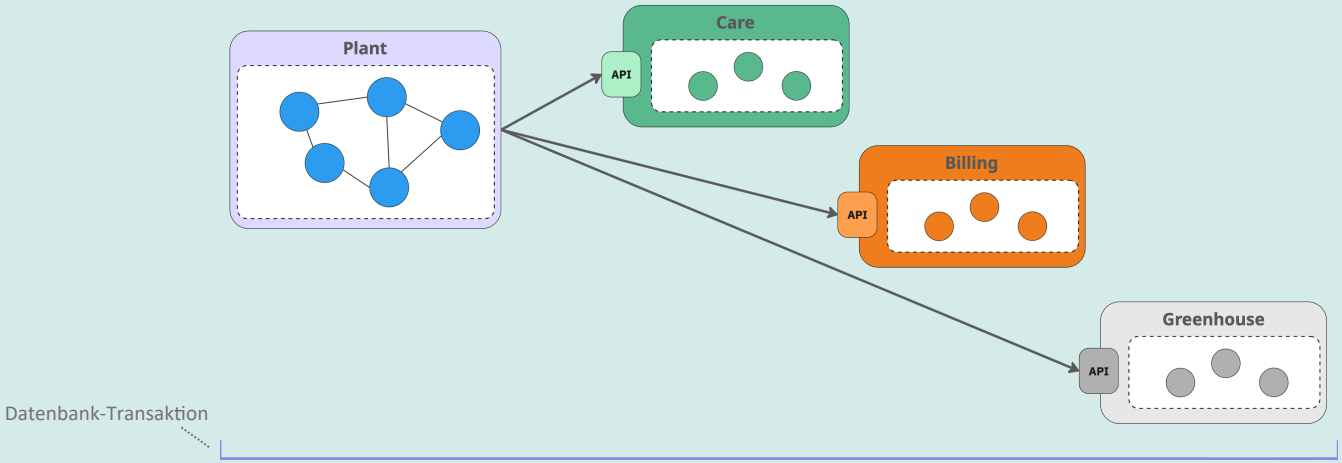
Kopplung von Modulen über Services

- Erweiterungen müssen an zwei Stellen erfolgen
- Tests müssen angepasst werden (Beans in Services!)
- Verletzung des Open-Closed-Prinzips



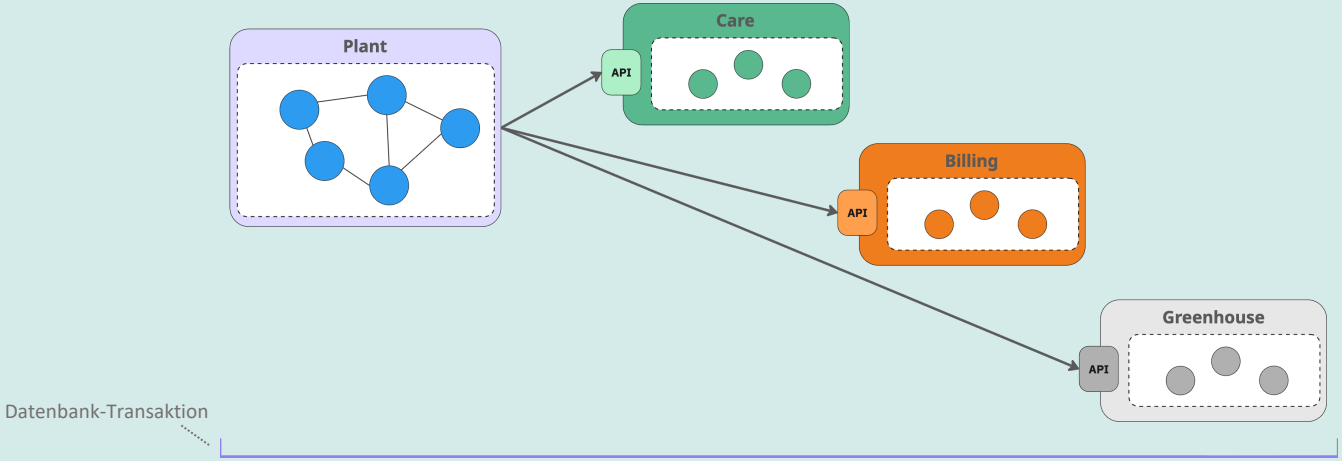
Datenbank-Transaktion

- Komplette Verarbeitung in einer Transaktion
- Garantiert Konsistenz



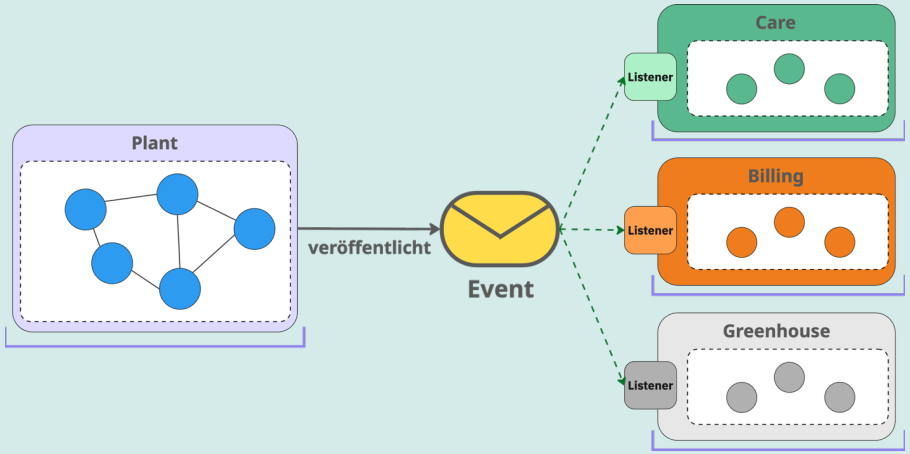
Datenbank-Transaktion

- Komplette Verarbeitung in einer Transaktion
- Garantiert Konsistenz
- Kann aber Ressourcen-Problem werden



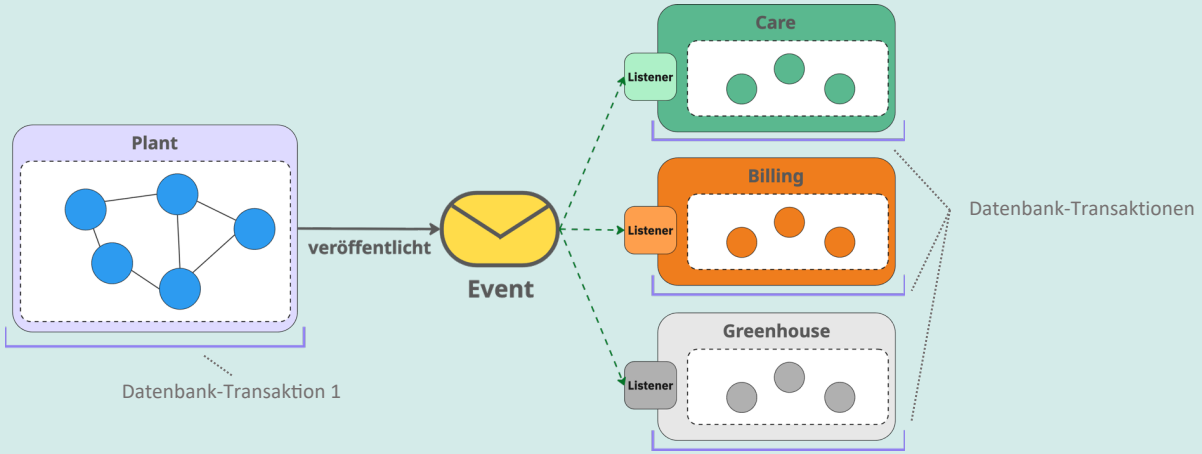
Entkopplung über Events

- Module veröffentlichen fachliche Events (Spring built-in Event Registry)
- Neue Consumer können jederzeit hinzugefügt werden



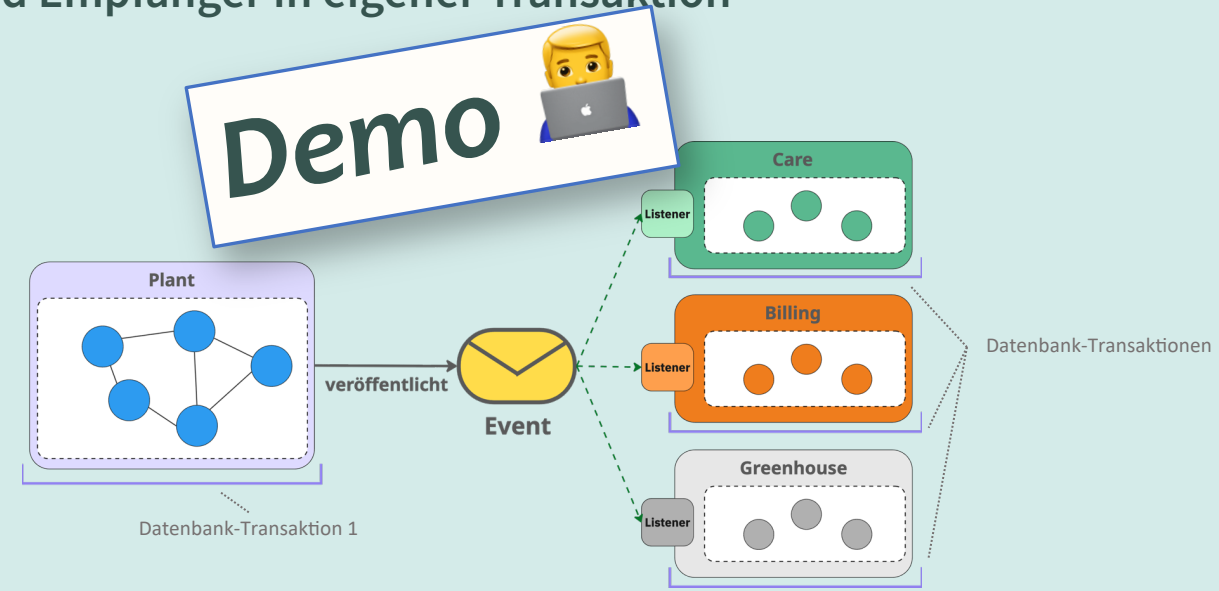
Entkopplung über Events

- Abarbeitung kann asynchron und parallel erfolgen
- Sender und Empfänger in eigener Transaktion



Entkopplung über Events

- Abarbeitung kann asynchron und parallel erfolgen
- Sender und Empfänger in eigener Transaktion



NILS HARTMANN
https://nilshartmann.net

Vielen Dank!

Slides & Code: https://spring.schule/sas2026-spring-modulith

Fragen und Kontakt:

nils@nilshartmann.net

https://nilshartmann.net/kontakt

